

API Spec: PubMed Custom Publication Retrieval API

TL;DR

A custom backend API that automatically queries the NCBI PubMed repository (via E-utilities `esearch` + `efetch`) to identify publications that resulted from completed Wake Forest SOM EHR data requests. The API ties PMIDs directly to REDCap Record IDs, populates the PI Profile's publication metrics, and drives the ROI scoring engine. Not real-time: it runs post-completion for closed requests, via a one-time historical backfill at deployment, and on a bi-monthly re-run for all requests 2–10 years old by Date of Request. Authenticated by a registered NCBI API key stored as an environment variable. Rate limit: 10 requests/second.

Goals

Business Goals

- Automate the manual, high-effort PubMed workflow documented in the DIVE ROI Workflow PDF; eliminate ad-hoc, non-repeatable lookups.
- Accurately and automatically populate PI Profile publication count and publication rate (percent of requests resulting in publications) from primary PubMed data.
- Feed the ROI scoring engine with normalized, deduplicated, structured publication data to power Publication Impact and Research Output scoring.
- Build a longitudinal, queryable dataset of Record ID–PMID mappings for future ML training and system improvement.

User Goals

- Data managers see complete, up-to-date publication lists and metrics in PI Profiles with no manual PubMed searching required.
- Zero- and filtered-result states are clear: a "No publications found" message with context appears for requests that return no publications, distinguishing no results from query failures or system gaps.
- The entire workflow becomes auditable, and output is reliably linked to individual requests for compliance and longitudinal analysis.

Non-Goals

- The API does not provide or support real-time publication searches for open or in-progress requests; only closed/completed REDCap records trigger queries.
 - The API does not score journal prestige, compute RCR/citations, or author metrics—those are handled by independent modules (CrossRef/Journal Matching, iCite, etc.).
 - The API does not serve as a direct replacement for iCite; instead, it serves as the input layer for iCite's downstream enrichment.
-

User Stories

- **Backend System**
 - As the backend, I want to query PubMed for each closed request using PI name, affiliation, and project title keywords, so that I can identify and store resulting publications.
 - As the backend, I want to log when no PubMed results are found or all are filtered out by author/year, so that data managers distinguish between truly unpublished work and query issues.
 - As the backend, I want to store Record ID–PMID matches in a many-to-many junction table, so that PI-level deduplication and reporting are always accurate.
 - **Data Manager**
 - As a data manager, I want to see "No publications found" clearly on PI Profiles rather than a blank field, so I know the system searched and found nothing rather than not running.
-

Functional Requirements

- **Input Parameters (Priority: High)**
 - Accepts: REDCap Record ID (unique), PI name in 'Last, First' format, Year Completed (int), Department (optional), Project Title (optional).
 - Inputs drawn directly from REDCap; no user-supplied overrides required.
- **Query Construction (Priority: High)**
 - PI last name extracted from input for NCBI [Author] field.
 - Institutional constant: Wake Forest[Affiliation].
 - Keyword extraction: Remove standard English + custom stopwords (study, analysis, assessment, outcomes, impact, effect, among, following, patients, data, using, based, related, associated), select two longest remaining words; fallback to Author + Affiliation only if none remain.
 - Year filter: Range filter using Year Completed + 2 (inclusive) as minimum, 3000 as max, via [dp].
 - Full query: LastName[Author] AND Wake Forest[Affiliation] AND (keyword1) AND (keyword2) AND MinYear:3000[dp].
- **NCBI E-utilities Integration (Priority: High)**
 - Use `esearch` to retrieve up to 100 PMIDs (`retmax=100`) in JSON; include `api_key` in all requests.
 - 0.1 second enforced delay between calls (matches API-key rate limit).
 - Use `efetch` in batch mode to retrieve XML for all returned PMIDs.
 - Implement exponential backoff on HTTP errors (429, 500, 503), up to three retries.
- **Post-Fetch Filtering (Priority: High)**

- **Filter 1:** Publication year (from PubDate XML) must be $\geq \text{Year Completed} + 2$.
- **Filter 2:** PI last name (case-insensitive) must appear in the [AuthorList] of the article XML.
- All filtering occurs post-fetch. Articles must pass both filters to be retained.
- **Result Output (Priority: High)**
 - On success, output: Record ID, PMID (string), DOI (string — extracted from PubMed XML ArticleId element where IdType='doi'; stored as NULL if not present), Year Published (integer), Journal (ISOAbbreviation preferred, Title fallback), Grants (comma-delimited GrantID/Acronym/Agency/Country).
 - *Note:* DOI serves two purposes — (a) primary input to CrossRef Journal Matching API for ISSN resolution, eliminating redundant CrossRef calls; (b) clickable article link in UI formatted as <https://doi.org/{DOI}> labeled 'View Article', opens in new tab.
 - Output is written to a normalized table and exposed via downstream queries.
- **Deduplication (Priority: High)**
 - Within-request: Maintain set of seen PMIDs to prevent duplicates.
 - Cross-request: Store all Record ID–PMID matches in a request_publications junction; all publication counts for PI are computed by `COUNT(DISTINCT PMID)`.
- **Zero-Result Handling (Priority: High)**
 - Distinguish:
 - a. `NO_RESULTS`: `esearch` yields empty idlist.
 - b. `FILTERED_OUT`: PMIDs returned by `esearch`, but none pass filters.
 - Both states are logged and surfaced in UI; no auto-query relaxation.
- **Historical Backfill (Priority: High)**
 - One-time script at deployment runs for ALL closed requests (no year cutoff), with correct filtering, deduplication, and logging.
 - Script is idempotent; safe for re-runs via `INSERT IGNORE` or equivalent.
- **Ongoing Trigger (Priority: High)**
 - **Trigger 1 — Initial Closure Run:** Fires automatically when a REDCap record transitions from open (`date_complete IS NULL`) to closed (`date_complete IS NOT NULL`) on upload. Runs once per record at closure. Primary mechanism for capturing publications from newly completed requests. Also available as a manual trigger per Record ID.
 - **Trigger 2 — Historical Backfill (Deployment Only):** One-time script run at deployment processing all REDCap records where `date_complete IS NOT NULL` regardless of completion year. Identical query construction, filtering, deduplication, and junction table logic as Trigger 1. Idempotent — safe to re-run using `INSERT IGNORE` or `ON CONFLICT DO NOTHING`.
 - **Trigger 3 — Bi-Monthly Re-Run for Active Publication Window:** Runs automatically every 2 months. Processes ALL REDCap records — open

or closed — where the Date of Request field from the REDCap export falls within a 2–10 year window from today: today minus 10 years <= Date of Request <= today minus 2 years. The 2-year floor eliminates computational waste on requests too recent to have publications (given Year Completed + 2 filter). The 10-year ceiling eliminates waste on requests old enough that net-new publications are highly unlikely. Intentionally uses Date of Request — not `date_complete` — as the window anchor, so publication searches run correctly even if the team has not marked a request complete in REDCap. Date of Request is the authoritative REDCap field for this calculation. Net-new PMIDs appended using `INSERT IGNORE` or `ON CONFLICT DO NOTHING` — existing records never overwritten.

- **Multi-PMID Per Request (Priority: High)**
 - A single REDCap request record is expected to generate more than one PMID — a research project often produces multiple publications over its lifetime (primary outcomes paper, secondary analysis, follow-up study). The system must never stop at the first matched PMID per request. All PMIDs passing author and year filters are stored on every trigger run. Trigger 3 exists specifically to capture PMIDs appearing in PubMed after the initial closure run, ensuring the full publication output of each project is accumulated over time. The `request_publications` junction table primary key on (record_id, pmid) ensures each unique PMID is stored once per request regardless of how many trigger runs process it.
-

User Experience

While a backend API with no user interface, key touchpoints include:

Entry Point & First-Time User Experience

- No user action required to trigger API; it is invoked automatically post-completion, via backfill, and bi-monthly for the active publication window.
- Onboarding: Data managers are notified that publication metrics now appear in PI Profiles — no more manual PubMed searching.

Core Experience

- **Step 1:** REDCap upload or automated status update sets request as closed.
 - System triggers PubMed API with input parameters from request.
 - Query construction is silent; user not involved.
- **Step 2:** API performs query, fetches, filters, and stores results.
 - Publication metrics update in PI Profile.
 - If no publications, a "No publications found" message, with context ("no PubMed results" vs. "all filtered out"), is displayed under the request.
- **Step 3:** Matched articles in PI Profile list include: PMID (hyperlink to <https://pubmed.ncbi.nlm.nih.gov/{PMID}/>, opens new tab), DOI link formatted as <https://doi.org/{DOI}> labeled 'View Article' (opens new tab, hidden if DOI is

NULL), Year Published, Journal name, and Grants string. When DOI is NULL, only the PubMed link is shown.

- **Step 4:** Team leads can review backfill status in a minimal admin panel.

Advanced Features & Edge Cases

- Historical backfill: Team Lead can monitor total processed, matched, and flagged records during backfill.
- Zero-result queries always return explicit result state; PI Profile never left blank for publication metrics.
- Manual trigger: Data managers/admins may re-run the API for any Record ID ad hoc (to catch late-indexed publications).
- If PubMed XML omits structured Year (older records), parser falls back to parsing `MedlineDate`.

UI/UX Highlights

- Explicit status for all requests (“matched”, “no results”, “filtered out”, “error”).
 - Linked PubMed PMIDs prevent copy-paste errors.
 - PI Profile always reflects the latest API result or error, ensuring clarity for downstream scoring.
-

Narrative

For years, DIVE ROI data managers meticulously searched PubMed for every completed EHR data request: copying PI names, searching with keywords, scanning for Wake Forest affiliations, and then manually pasting matching PMIDs into spreadsheets. Each publication list was prone to omission, difficult to audit, and tedious to update. With the PubMed Custom Publication Retrieval API, this manual drudgery disappears. The system now runs these queries programmatically on all closed requests — and, at launch, sweeps through years of history overnight, backfilling PI Profiles and scoring records with accuracy, transparency, and auditability. Data managers arrive the next morning to fully populated PI Profiles: every publication linked, counted, and ready for reporting, with no more time wasted on ad-hoc PubMed lookups.

Success Metrics

- **Backfill completion rate:** 100% of historical closed records processed with a logged status.
- **API enrichment success rate:** $\geq 85\%$ of closed requests yield a logged status (matched, no_results, filtered_out) within the scheduled processing window.
- **False positive rate:** Spot check 20 matched publications; target $<5\%$ mismatches with actual PI publication records.
- **Zero silent failures:** Each API call logs a status for every request; no unsurfaced NULL results post-processing.

User-Centric Metrics

- % of PI Profiles with visible, non-blank publication metrics.

of manual PubMed lookups performed by data managers (should approach zero).

Business Metrics

- Reduction in manual hours spent on annual/quarterly publication reporting.
- % of publication records linked directly to REDCap requests post-backfill.
- Longitudinal dataset growth rate for future ML training (e.g., # of PMID–Request ID links per month).

Technical Metrics

- 0% API key quota overruns.
- API error rate after rollout: <2%.
- API query-to-response median latency: <2 seconds per Record ID (across all retries).

Tracking Plan

- For each API call:
 - Record ID processed.
 - Query string used.

of PMIDs returned by esearch.

of PMIDs passing filters.

- Final status (matched/no_results/filtered_out/error).
 - API response time (ms).
 - Timestamp of processing.
- Log Trigger 3 bi-monthly re-runs separately from Trigger 1 closure runs — track: records in 2–10 year Date of Request window processed per cycle, net-new PMIDs appended per cycle, and records where no new PMIDs were found.

Technical Considerations

Technical Needs

- **APIs:** NCBI E-utilities (`esearch.fcgi`, `efetch.fcgi`) with authenticated API key usage.
- **Backend module:** Python/Node/other (as appropriate) to manage query logic, filtering, deduplication, rate limiting, and retries.
- **Database:** Persistent junction table `request_publications` (`record_id` VARCHAR, `pmid` VARCHAR, `doi` VARCHAR, `matched_on_date` DATE,

query_string TEXT, journal_iso VARCHAR, journal_full VARCHAR, year_published INTEGER, grants TEXT) with PK on (record_id, pmid).

Integration Points

- **REDCap integration:** Input parameters, triggers on completed requests.
- **PI Profile/ROI Scoring Engine:** Downstream consumers of structured output.
- **iCite enrichment pipeline:** All stored PMIDs available for RCR/citation enrichment; must be refreshed for all PMIDs periodically.

Data Storage & Privacy

- API key stored as `NCBI_API_KEY` environment variable only—never committed to code.
- All Record ID–PMID matches enforced deduplicated at DB-level.
- Data processed/stored: No PHI/PII outside normal workflow; only publication metadata and institutional project metadata.

Scalability & Performance

- Expected load: Hundreds (not thousands) of closed requests at initial backfill, tens per week ongoing.
- Rate limiting enforced: 10 requests/sec with API key; 0.1s enforced minimum delay.
- Batch fetch timeouts: Per-batch timeout (max 30s), with retry.

Potential Challenges

- PI last name disambiguation (e.g., "Smith")—common names with Wake Forest affiliation may still yield false positives; regular QA spot checks needed.
 - PubDate MedlineDate formats may require robust XML parsing logic.
 - Batch fetch risk: network failures or slow calls can cause timeouts—need robust retry and logging.
 - Backfill idempotency critical; no duplicate DB entries on rerun.
 - Trigger 3 bi-monthly re-run processes all records in the 2–10 year Date of Request window regardless of completion status. Estimate total record volume in this window before deployment and batch across multiple nights if needed.
-

Milestones & Sequencing

Project Estimate

- **Small:** 3–5 days development + 0.5 day manual QA/spot-check.

Team Size & Composition

- **Small Team:** 1 backend developer (API logic, DB schema); QA/data manager time for spot-checking results.

Suggested Phases

Phase 1: NCBI Integration & Query Construction (Day 1–2)

- Deliverables: Backend dev implements esearch/efetch integration, API-key based auth, rate limiting, exponential backoff, and robust query construction (with stopword file as config).
- Dependencies: NCBI API key (configured as env var), sample REDCap export.

Phase 2: Filtering, Output Parsing, Deduplication (Day 2–3)

- Deliverables: Parse full XML, output all required PubMed fields; implement within-request deduplication and create DB table with unique constraints for cross-request dedupe.
- Dependencies: XML parser for PubMed output.

Phase 3: Result Logging, Historical Backfill & Ongoing Trigger (Day 3–4)

- Deliverables: Implement explicit status/result logging (matched, no_results, filtered_out); build one-time historical backfill + status admin panel; set up automatic API trigger on REDCap closure.
- Dependencies: REDCap export for all historical closed requests.

Phase 4: QA, Spot Check, and iCite Sync (Day 4–5)

- Deliverables: Spot check 20 known matched publications; validate output, error handling, and logging; confirm all PMIDs eligible for iCite enrichment cycle.
- Dependencies: Data manager available for review.

Developer Callouts

1. NCBI API key must be stored as an environment variable (`NCBI_API_KEY`); do not hardcode anywhere in the codebase.
2. If PubMed XML `PubDate` element uses `MedlineDate` (e.g., "Winter 2019") instead of structured `Year`, parser must handle both XML formats robustly.
3. Backfill script must be fully idempotent: re-runs never create duplicate DB entries.
4. Downstream iCite enrichment cycle must refresh *all* PMIDs in `request_publications` on each job, regardless of publication date (RCR may change over time).
5. The custom research stopword list must be a text file in the config directory; never hardcoded, so it can be updated without code deploy.
6. Three triggers govern when the API runs: (a) Trigger 1 on request closure; (b) Trigger 2 one-time deployment backfill; (c) Trigger 3 bi-monthly re-run for all records where Date of Request is 2–10 years ago. Trigger 3 uses Date of Request — not `date_complete` — ensuring publications are found even for requests not yet marked complete.
7. Estimate Trigger 3 record volume before deployment — calculate records in the REDCap database with Date of Request in the 2–10 year window and confirm the bi-monthly job completes overnight. Batch across multiple nights if volume is large.

8. Extract DOI from PubMed XML at efetch time from ArticleId element where IdType='doi'. Store in request_publications immediately — it is the primary input to the CrossRef Journal Matching API and drives the 'View Article' link in the UI. Store as NULL when not present; older publications frequently lack a DOI in PubMed.
-